

Networks Lab Assignment 3 - Application #7

Point-of-Sale Terminal (Client-Server, TCP, C)

Full team briefing. Anyone (or any AI assistant) can pick up from this document alone.

How to use this document

- Read Sections 1 to 6 (shared context). Then read only your own person section in Section 7.
- If you use an AI assistant, give it this whole PDF and start with the prompt in Section 11, filling in your name and part.
- Do not skip the shared contract (Section 5). All four parts only work together if everyone follows it exactly.

1. Course and assignment facts

Item	Detail
Course	S3104L / CS3104L: Networks Lab (July-November 2026)
Assignment	Assignment 3: Socket Programming
Released	30 August 2026
Deadline	11:55 PM, Monday 28 September 2026
Group	3-4 members. Our group has 4 members (see Section 4). Group number: _____ (fill in)
Language	C or C++ only. We are using C.
Submission	One zip named after the group number, for example Group_4.zip. Max 1 MB. Only the group leader uploads (Moodle/MS Teams). Source code files only.
Evaluation	Offline viva-voce in the lab. Each member must explain the code and run it in front of the evaluator.
Plagiarism	Plagiarism tool is used. Copying gives NEGATIVE marks equal to the maximum marks. Write your own code and understand every line.
Reference book	Unix Network Programming Vol. 1, W. Richard Stevens (first few chapters)

2. What Application #7 is (plain language)

We are building a cash register over the network. There are two programs:

- **Client (the register):** connects to the server over TCP. The cashier types a product code and a quantity. The client sends it and shows the reply. When done, the cashier closes the session and sees the total bill.
- **Server:** looks up each product code in a text file (the product database). It replies with the price and name, or an error. It keeps a running total for each connected client. When a client closes, the server sends back that client's total.

Key requirements from the assignment:

- The server must be **concurrent**: several clients connected at the same time, each with its own separate total.
- IP address and port come from the **command line**. Do not hard-code the port.
- Command lines: `./client <server_ip> <server_port>` and `./server <server_port>`.

- Connections must be closed **gracefully** (socket released).
- When the server is stopped with **Ctrl+C**, it must release its open socket. Hint from the assignment: use a signal handler.
- A text (TXT) file is used as the database with the product code and item description.
- Make and state sensible assumptions where the assignment is unclear.

Message formats given in the assignment

Request (client to server): Request_Type, UPC-Code, Number

- Request_Type: 0 = item, 1 = close.
- UPC-Code: 3-digit unique product code. Only meaningful when Request_Type is 0.
- Number: how many of the item are being bought. Only meaningful when Request_Type is 0.

Response (server to client): Response_Type, Response

- Response_Type: 0 = OK, 1 = error.
- OK, after an item request: Response is of the requested item.
- OK, after a close request: Response is the total amount for all purchases by that client.
- Error: Response is a null-terminated string. The only allowed errors are exactly **"Protocol Error"** and **"UPC is not found in database"**.

3. Design decisions the whole team must follow

Topic	Decision
Transport	TCP, IPv4 (AF_INET, SOCK_STREAM)
Concurrency	fork() per client. Each child process has its own local total. Parent handles SIGCHLD to avoid zombies.
Message packing	Fixed-size structs (Section 5). Send and receive with helper functions that loop until all bytes are transferred.
Byte order	Every int field is converted with htonl() before sending and ntohl() after receiving. Both client and server do this.
Database file	products.txt in the server's working directory. One product per line: . Names have NO spaces (use Brown_Bread). Price is a decimal number.
Price and total	Stored as double. Printed with 2 decimals.
Quantity rule	Must be a positive integer. Zero or negative gives Protocol Error.
UPC rule	Must be between 100 and 999 (3 digits). Otherwise Protocol Error.
Unknown request type	Anything other than 0 or 1 gives Protocol Error.
After an error	Connection stays open. The client can try again.
After close	Server sends the total, then both sides close the socket.
Ctrl+C on server	SIGINT handler closes the listening socket and exits. SO_REUSEADDR is set so the port can be reused immediately.

4. Team split

Member	Owns these files	Summary
Abhinav Chouhan	server.c	Socket setup, accept loop, fork, signal handlers, calling the handler
Abhinav Rai	client.c	Connect, menu loop, send requests, print responses, input checking
Adithyan	protocol.h, common.c, db.c, products.txt	Shared contract, send_all / recv_all, database lookup, product data
Harshit	handler.c, Makefile, README.txt, testing, zip	Request logic, build, full test run, documentation, packaging

Final files in the zip: server.c, client.c, protocol.h, common.c, db.c, handler.c, products.txt, Makefile, README.txt. Do not include test files or compiled binaries.

5. Shared contract: protocol.h

Adithyan writes this file first and shares it. Everyone uses the same copy. Nobody changes it without telling the whole group.

```
#ifndef PROTOCOL_H
#define PROTOCOL_H
#include <stddef.h>

#define REQ_ITEM    0
#define REQ_CLOSE   1
#define RESP_OK     0
#define RESP_ERR    1
#define MSG_LEN     128

typedef struct {
    int request_type; /* 0 = item, 1 = close */
    int upc;          /* 3-digit code (ignored for close) */
    int number;       /* quantity (ignored for close) */
} Request;

typedef struct {
    int response_type; /* 0 = OK, 1 = error */
    char response[MSG_LEN]; /* "<price> <name>", total, or error text */
} Response;

/* common.c */
int send_all(int fd, const void *buf, size_t len); /* 0 ok, -1 fail */
int rcv_all(int fd, void *buf, size_t len);       /* 0 ok, -1 fail or closed */

/* db.c */
/* returns 1 if found, 0 if not found */
int lookup_product(const char *dbfile, int upc, double *price,
                  char *name, size_t namelen);

/* handler.c */
/* returns 1 if the connection should close after this reply, else 0 */
int handle_request(const Request *req, Response *resp, double *total);

#endif
```

Byte order rule: on the wire, request_type, upc, number and response_type are in network byte order. handle_request() always works with host-order values, so the server converts right after rcv and right before send. The response text is plain bytes and needs no conversion.

products.txt example format

```
101 Milk 45.50
102 Bread 30.00
103 Eggs 72.00
104 Butter 58.25
105 Rice_1kg 64.00
106 Sugar_1kg 48.00
107 Tea_250g 110.00
108 Biscuits 20.00
109 Soap 35.00
110 Toothpaste 89.50
```

6. How the pieces fit together

- **server.c** accepts a connection, forks a child. The child loops: rcv_all a Request, ntohs the fields, call handle_request(), htons the response type, send_all the Response.
- **handler.c** holds the business rules. It calls lookup_product() in db.c, updates the running total, and fills the Response.
- **db.c** reads products.txt and finds a product by code.

- **common.c** gives send_all and recv_all used by both server and client.
- **client.c** shows a menu, builds a Request, sends it, reads the Response, prints it.

Typical session:

```

Client                                     Server (child for this client)
connect -----> accept, fork, total = 0
send {ITEM, 101, 2} -----> lookup 101 -> Milk 45.50
                                     total += 91.00
<----- {OK, "45.50 Milk"}
send {ITEM, 999, 1} -----> not found
<----- {ERR, "UPC is not found in database"}
send {CLOSE, 0, 0} -----> reply total
<----- {OK, "91.00"}
close socket                             close socket, child exits

```

7. Individual work: exactly what each person does

7.1 Abhinav Chouhan: server.c

Goal: a concurrent TCP server that takes the port from the command line, serves each client in its own process, and shuts down cleanly on Ctrl+C.

Depends on: protocol.h, common.c (send_all, recv_all), handler.c (handle_request). Until those are ready, use a temporary stub handle_request that returns an OK response.

Steps

- 1. In main, check argc == 2. If not, print Usage: ./server <port> and exit. Read the port with atoi(argv[1]).
- 2. Create the listening socket: socket(AF_INET, SOCK_STREAM, 0). Check for -1.
- 3. Call setsockopt(fd, SOL_SOCKET, SO_REUSEADDR, ...) so restarting does not give 'Address already in use'.
- 4. Fill a sockaddr_in: sin_family = AF_INET, sin_addr.s_addr = htonl(INADDR_ANY), sin_port = htons(port). Call bind(), then listen(fd, 10). Check each call.
- 5. Keep the listening fd in a global variable so the signal handler can close it.
- 6. Register handlers with sigaction (not signal): SIGINT handler closes the listening fd, prints a short message, and calls _exit(0). SIGCHLD handler loops waitpid(-1, NULL, WNOHANG) while it returns greater than 0. Use SA_RESTART for SIGCHLD.
- 7. Main loop: connfd = accept(). If it returns -1 with errno == EINTR, continue. On other errors print and continue.
- 8. Print the client's IP and port (inet_ntop) for debugging.
- 9. fork(). In the child: close the listening fd, set double total = 0;, then loop as described in step 10, then close(connfd) and exit(0). In the parent: close(connfd) and go back to accept.
- 10. Child loop: recv_all(connfd, &req;, sizeof req). If it fails (client left), break. Convert req.request_type, req.upc, req.number with ntohl. Call done = handle_request(&req, &resp, &total). Convert resp.response_type with htonl. send_all the Response. If done is 1, break.

Done when

- ./server 5000 starts, and ./server with no argument prints usage.
- Two clients can be connected at once and get separate totals.
- Ctrl+C prints a message and the port can be reused immediately.
- ps aux | grep server shows no zombie (defunct) processes after clients disconnect.
- You can explain why fork() is used, what SO_REUSEADDR does, and why the parent closes connfd.

7.2 Abhinav Rai: client.c

Goal: a register program that connects to the server, lets the user enter purchases, and shows the server's replies.

Depends on: protocol.h and common.c (send_all, recv_all). It can be tested against Abhinav Chouhan's server with the stub handler.

Steps

- 1. In main, check argc == 3. If not, print Usage: ./client <server_ip> <port> and exit.

- 2. Create a socket. Fill `sockaddr_in` with `AF_INET`, `htons(atoi(argv[2]))`, and `inet_pton(AF_INET, argv[1], &addr.sin_addr)`. If `inet_pton` fails, print 'Invalid IP address' and exit.
- 3. `connect()`. If it fails, print the error with `perror` and exit.
- 4. Menu loop: '1) Buy item 2) Close and get total'.
- 5. Option 1: ask for UPC code and quantity with `scanf`. Check that `scanf` returned 2. If the user typed letters, clear the rest of the input line and ask again. Build a Request with `request_type = REQ_ITEM`.
- 6. Option 2: build a Request with `request_type = REQ_CLOSE`, `upc = 0`, `number = 0`.
- 7. Convert all three fields with `htonl`. `send_all` the Request. `recv_all` a Response. Convert `response_type` with `ntohl`. Make sure the response text is null-terminated before printing (set the last byte to 0).
- 8. If `response_type` is OK after an item request, print Price and item: <response text>. If it is an error, print the error text and stay in the loop.
- 9. If OK after close: print Total amount: <response text>, `close(sock)`, `exit(0)`.
- 10. If `send_all` or `recv_all` fails at any time, print 'Server disconnected', close the socket, and exit.

Done when

- Connects with `./client 127.0.0.1 5000`.
- Letters typed where a number is expected do not crash or loop forever.
- A wrong UPC shows the error and you can keep entering items.
- Close prints the total and the program exits cleanly.
- Killing the server while the client is running makes the client print a message and exit, not hang.

7.3 Adithyan: protocol.h, common.c, db.c, products.txt

Goal: provide the shared contract and the low-level helpers the other three depend on. You unblock everyone, so do `protocol.h` first and share it the same day.

Steps

- 1. Create `protocol.h` exactly as in Section 5 and send it to the group first.
- 2. **common.c: `send_all(fd, buf, len)`**. Loop with `send()`. Track bytes sent. If `send` returns -1 and `errno` is `EINTR`, retry. Any other -1 returns -1. Use the `MSG_NOSIGNAL` flag so a closed peer does not kill the process with `SIGPIPE`. Return 0 when all `len` bytes are sent.
- 3. **common.c: `recv_all(fd, buf, len)`**. Loop with `recv()`. If it returns 0, the peer closed, so return -1. If -1 with `EINTR`, retry. Any other -1 returns -1. Return 0 when all `len` bytes have arrived. Reason: TCP is a byte stream and one `recv` may return only part of the struct.
- 4. **db.c: `lookup_product(dbfile, upc, &price, name, namelen)`**. `fopen` the file (return 0 if it cannot be opened). Loop with `fscanf(f, "%d %63s %lf", &code, tmpname, &p)` while it returns 3. If `code == upc`, copy the name with `strncpy`, set `*price`, `fclose`, return 1. At end of file, `fclose` and return 0.
- 5. **products.txt:** at least 10 products with unique 3-digit codes, names with no spaces, prices with up to 2 decimals (see Section 5).
- 6. Write a small throwaway test program (`test_db.c`) that calls `lookup_product` for a found code and a missing code. Do not put it in the final zip.

Done when

- The other three can compile against `protocol.h` without changes.

- send_all and recv_all move a 132-byte Response correctly even if the network splits it.
- lookup_product returns 1 with the right price and name for a valid code and 0 for an unknown code.
- You can explain why recv_all needs a loop and what recv returning 0 means.

7.4 Harshit: handler.c, Makefile, README.txt, testing, zip

Goal: own the business rules, make the project build with one command, prove it works, document it, and package it.

Part A: handler.c

Function: `int handle_request(const Request *req, Response *resp, double *total)`. It must use host-order values. It returns 1 if the connection should close after replying, otherwise 0.

- Start with `memset(resp, 0, sizeof *resp)` so strings are always null-terminated.
- If `req->request_type == REQ_ITEM`: if number is 0 or less, or upc is not between 100 and 999, set `response_type = RESP_ERR`, text "Protocol Error", return 0.
- Otherwise call `lookup_product("products.txt", req->upc, &price, name, sizeof name)`. If it returns 0, set `RESP_ERR` and "UPC is not found in database", return 0.
- If found: `*total += price * req->number`. Set `RESP_OK` and `snprintf(resp->response, MSG_LEN, "%.2f %s", price, name)`. Return 0.
- If `request_type == REQ_CLOSE`: set `RESP_OK` and `snprintf` the total with "%.2f". Return 1.
- Any other `request_type`: `RESP_ERR` and "Protocol Error". Return 0.

Part B: Makefile

```
CC=gcc
CFLAGS=-Wall -Wextra -g

all: server client

server: server.c common.c db.c handler.c protocol.h
    $(CC) $(CFLAGS) -o server server.c common.c db.c handler.c

client: client.c common.c protocol.h
    $(CC) $(CFLAGS) -o client client.c common.c

clean:
    rm -f server client
```

Recipe lines (gcc and rm) must start with a TAB character, not spaces. The PDF shows them indented with spaces, so retype the indent as a real Tab.

Part C: Testing (after everyone's code is merged)

Test	Expected result
make	Builds with no errors. Fix or report warnings.
Buy code 101 quantity 2	Client prints the price and name. Total increases by 2 x price.
Buy an unknown code, such as 999	Prints 'UPC is not found in database'. Client stays connected.
Quantity 0 or negative	Prints 'Protocol Error'. Client stays connected.
Non-numeric input at the client	Client re-prompts and does not crash.
Two or three clients at once	Each has its own separate running total.
Close	Prints the correct total. Client exits. Server child exits.
Kill a client with Ctrl+C mid-session	Server keeps running, no crash, no zombie.
Kill the server while a client is connected	Client prints a message and exits.

Test	Expected result
Ctrl+C on the server, then restart on the same port	No 'Address already in use'.
ps aux grep server	No defunct (zombie) processes.
Run with wrong or missing arguments	Both programs print usage and exit.

Run commands:

```
./server 5000
./client 127.0.0.1 5000
```

Part D: README.txt

- How to compile (make) and run (server and client commands).
- Message formats (Request and Response), with the byte-order rule.
- Files and who wrote each.
- Assumptions: prices are decimals (double); quantity is a positive integer; UPC is 3 digits (100-999); products.txt is static and read on each request; product names contain no spaces; each client has its own total because each client is served by a separate forked process.

Part E: Packaging

zip Group_<N>.zip server.c client.c common.c db.c handler.c protocol.h products.txt Makefile README.txt

Replace <N> with the group number. Check the zip is under 1 MB and contains no binaries. Hand it to the group leader to upload.

8. Timeline (deadline 28 Sept 2026, 11:55 PM)

When	What
Day 1	Adithyan shares protocol.h. Everyone starts their part.
By 25 Sept	All parts written, merged, and compiling.
26-27 Sept	Harshit runs the full test list. Fix bugs. Everyone reads all files.
28 Sept, morning	Final zip made. Group leader uploads. Do not wait for the last hour.

9. Viva preparation (all four members)

Every member must be able to explain every file, not only their own. Swap files and read each other's code. Likely questions:

- Why TCP and not UDP for this application?
- What do socket, bind, listen, accept and connect each do?
- Why does the server use fork()? What would happen without it? What is the alternative (threads)?
- Why does the child close the listening socket and the parent close the connected socket?
- Why do send_all and recv_all use loops? What does recv returning 0 mean?
- Why htonl and ntohl? What is network byte order?
- What does SO_REUSEADDR do?
- What happens on Ctrl+C on the server? Why sigaction and why close the listening socket?
- What is a zombie process and how does the SIGCHLD handler prevent it?
- How does each client get its own total?
- What errors can the server return, and when?
- What assumptions did you make?
- Show a demo: two clients at once, a wrong code, close.

10. Rules to remember

- Write your own code. The plagiarism tool gives negative marks equal to the full marks. If an AI helps you, use it to understand and review, then write and be able to explain your own version line by line.
- C or C++ only. Port and IP from the command line. No hard-coded port.
- Only the two exact error strings are allowed in error responses.
- Keep the zip under 1 MB and named after the group number.

11. Prompt to give an AI assistant

Copy this, fill in the two blanks, and attach this PDF:

I am _____ (my name), a member of a 4-person group doing Networks Lab Assignment 3, Application #7 (Point-of-Sale Terminal over TCP in C). The attached PDF has the full context, the shared protocol.h contract, and the split of work. My part is: _____ (my file(s) from Section 7).

Please:

1. Explain my part in simple language and how it connects to the others.
2. Guide me step by step so I can write it myself (I must be able to explain every line in a viva, and the code must be my own).

3. Review my code when I paste it: find bugs, check it against the shared contract, and suggest tests.
Do not change protocol.h or the byte-order and error-string rules.